Controllo dei diritti dei soggetti

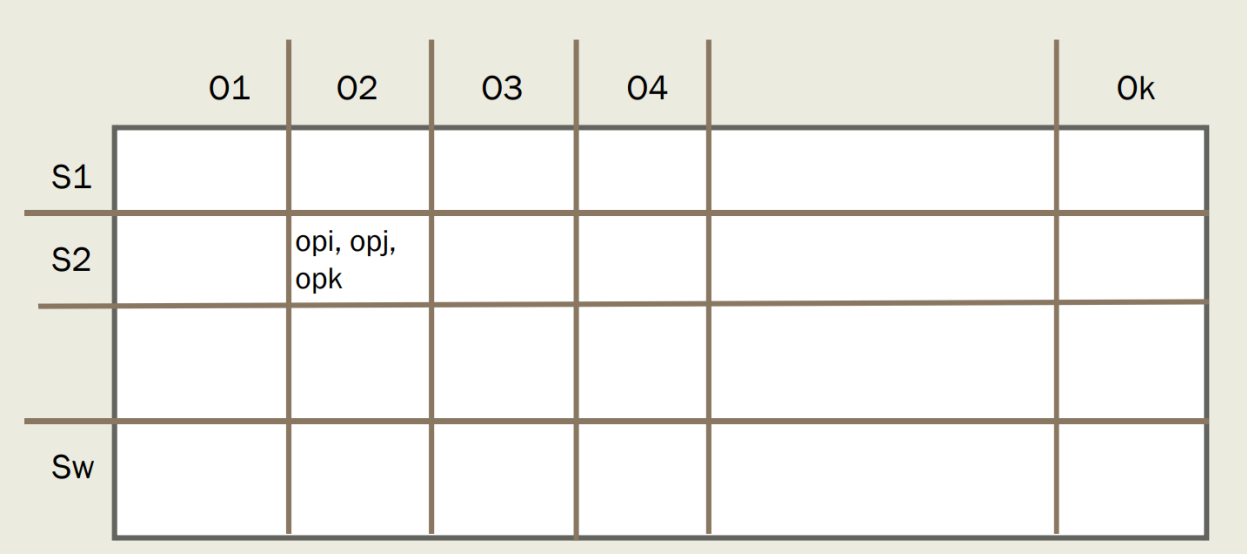
Description

Controllo degli accessi

Stiamo parlando della gestione della matrice di controllo degli accessi composta da soggetti, oggetti e diritti. Questo tipo di gestione è tipica del DAC (Discretionary Access Control).

Una matrice di controllo degli accessi può esistere a livello di:

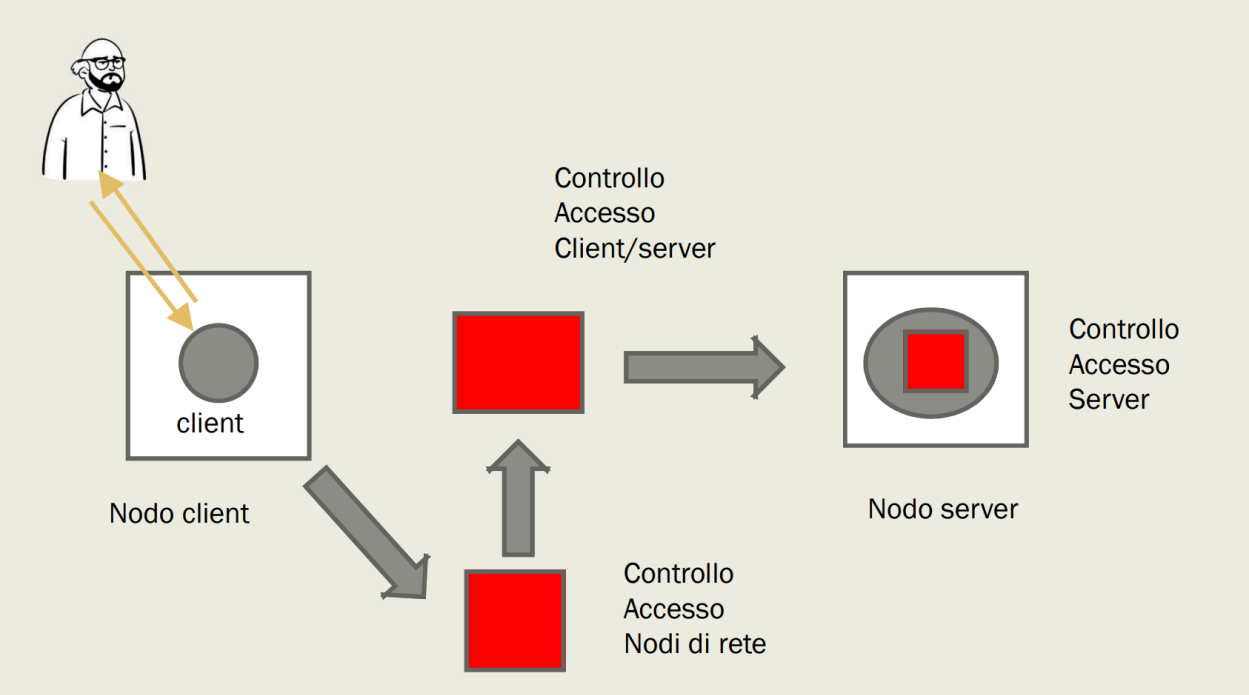
- **Sistema operativo**: per la protezione di zone di memoria, file, dispositivi di I/O, porte dei vari protocolli.
- **Applicazione**: per la gestione specifica di un'applicazione, ad esempio un database o un web server.



Controllo degli accessi in SO e applicazioni

In generale, la matrice del SO determina quali utenti (client) possono interagire con un'applicazione (server). Questo controllo avviene attraverso meccanismi come le porte o i canali di comunicazione.

- Per i client autorizzati all'interazione, la matrice di controllo degli accessi del server determina quali operazioni ciascun client può richiedere.
- Se un client non può interagire con un certo server, è consigliabile **impedire l'interazione al livello più basso possibile**, evitando di ricevere la richiesta e poi scartarla nel server.
- **Overhead**: scartare una richiesta a livello di server ha un costo elevato e può facilitare attacchi DoS.



Controllo degli accessi in astratto

Schema:

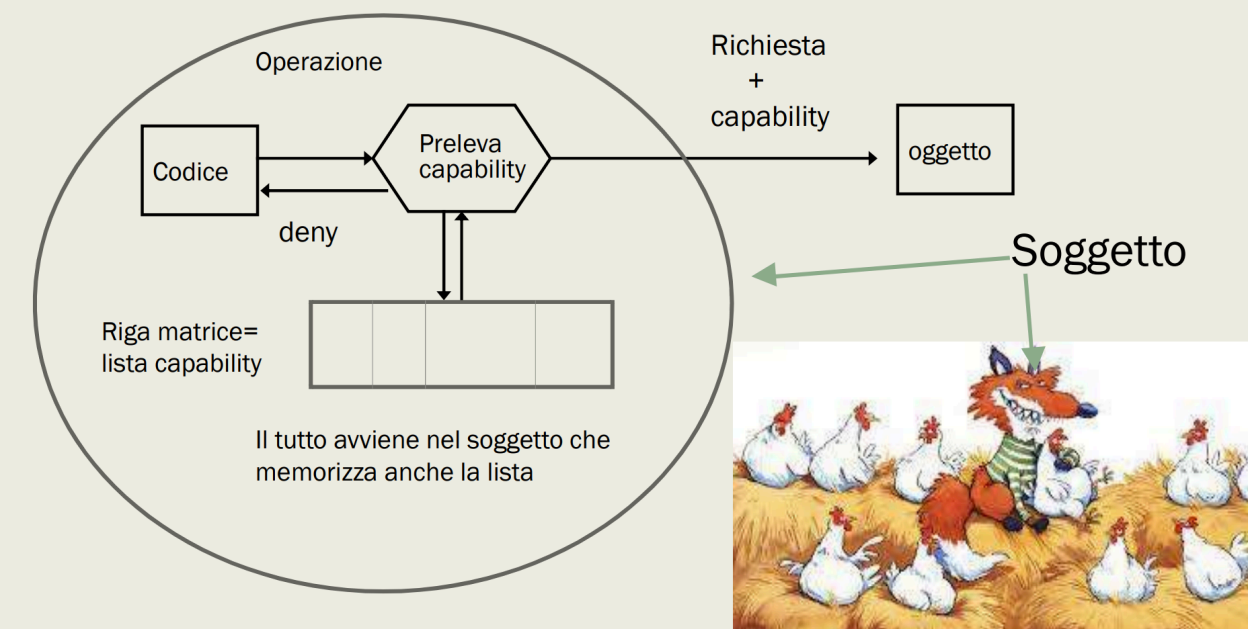
1. **Soggetto** invia una **richiesta**
2. **Reference monitor** valuta la richiesta
3. Se consentita (`allow`), il soggetto accede all'**oggetto**
4. Se negata (`deny`), l'accesso viene bloccato

La matrice di controllo degli accessi centralizzata può essere inefficiente, specialmente nei sistemi operativi. È necessario parallelizzare il controllo

decomponendo la matrice.

Due principali approcci:

- **Righe = Capability**
- **Colonne = ACL (Access Control List)**



Controllo degli accessi con ACL:

```
class ACL:
    def __init__(self):
        self.permissions = {}  # Dizionario che associa utent
i a permessi

    def grant_permission(self, user, permission):
        if user not in self.permissions:
            self.permissions[user] = set()
        self.permissions[user].add(permission)
```

```
    def revoke_permission(self, user, permission):
        if user in self.permissions and permission in self.pe
rmissions[user]:
            self.permissions[user].remove(permission)

    def check_permission(self, user, permission):
        return user in self.permissions and permission in sel
f.permissions[user]


# Esempio di utilizzo
acl = ACL()
acl.grant_permission("alice", "read")
acl.grant_permission("bob", "write")

print(acl.check_permission("alice", "read"))  # True
print(acl.check_permission("alice", "write"))  # False
```

Capability-based Access Control (CBAC)

Le capability memorizzano le informazioni nel soggetto, che le presenta all'oggetto come una sorta di "biglietto" che autorizza l'operazione.

- Esempio: protezione dei segmenti di memoria, dove la tabella di traduzione degli indirizzi include bit che indicano le operazioni consentite.
- Ogni puntatore a un oggetto diventa un **puntatore protetto** con le operazioni consentite.
- I puntatori devono essere protetti: una soluzione è l'uso della **crittografia** per cifrarli e decifrarli solo al momento dell'uso.

Controllo degli accessi con Capability:

```
import uuid
```

```python
class Capability:
    def __init__(self, resource, permissions):
        self.id = uuid.uuid4()
        self.resource = resource
        self.permissions = permissions  # Lista di permessi

    def check_access(self, operation):
        return operation in self.permissions


# Creazione di una capability per leggere un file
read_file_capability = Capability("file.txt", ["read"])

# Verifica del permesso
if read_file_capability.check_access("read"):
    print("Accesso consentito per la lettura del file")
else:
    print("Accesso negato")
```

Esempi di architetture basate su capability:

- Plessey PP250, CAP: tutti i puntatori sono capability.
- Capability protette da hardware/OS mediante registri specializzati.

Implementazione distribuita:

- Le capability devono essere trasferite in rete e protette con crittografia.
- Un server assegna un **segreto** e utilizza una funzione one-way non invertibile:
  - `capability = check digits = f (SECRET, protected fields)`
  - Quando viene presentata al server, viene verificata con `f (SECRET, protected fields) = check digits`.
- Problemi:
  - **Protezione** da tampering e furto.

- **Controllo della propagazione**: solo il gestore può creare capability.
- **Delega**: un soggetto può trasmettere una capability, ma questo introduce problemi di sicurezza.
- **Revoca**: difficile se sono state delegate capability.

Esempio: Stack basato su capability per MIPS:

- Implementato come coprocessore per capability.
- 32 registri per capability.
- Non esiste un'istruzione per estendere i diritti, solo per restringerli.

Armv8.5-A Memory Tagging:

- Ogni allocazione di memoria ha un **tag**.
- L'accesso deve avvenire con un puntatore con lo stesso tag.
- Se i tag non corrispondono, il SO può bloccare o segnalare l'accesso errato.

Access Control List (ACL)

Le ACL memorizzano la matrice di controllo degli accessi **nell'oggetto** da proteggere.

- L'implementazione dell'oggetto prevede informazioni per controllare le richieste.
- **Esempio tipico**: Router, che instradano pacchetti in base a regole di accesso.
- Ogni linea di un router ha una **lista di regole** per decidere quali pacchetti possono passare.
- Es. ACL su un'interfaccia di router:
  - `131.114.*.* → route`
  - `131.4.5.6 → route`
  - `131.4.*.* → drop`

L'ordine delle regole è **fondamentale**: prima vengono valutati i permessi, poi i divieti.

Routing in Linux: iptables

Linux gestisce il controllo degli accessi ai pacchetti di rete tramite `iptables`, che definisce:

- **Input chain**: regole per pacchetti in ingresso.
- **Output chain**: regole per pacchetti in uscita.
- **Forward chain**: regole per pacchetti che attraversano il nodo.

```
# Blocca tutto il traffico in ingresso dalla rete 192.168.
1.0/24
sudo iptables -A INPUT -s 192.168.1.0/24 -j DROP

# Permette il traffico SSH in ingresso solo dall'IP 192.16
8.1.100
sudo iptables -A INPUT -p tcp --dport 22 -s 192.168.1.100
-j ACCEPT

# Permette tutto il traffico in uscita
sudo iptables -A OUTPUT -j ACCEPT

# Visualizza le regole impostate
sudo iptables -L -v
```

Azioni possibili:

- **Drop**: pacchetto scartato.
- **Route**: pacchetto inoltrato.
- **Goto/Return**: passaggio a un'altra catena.
- **Queue**: invio a codice utente.
- **Log**: registrazione.

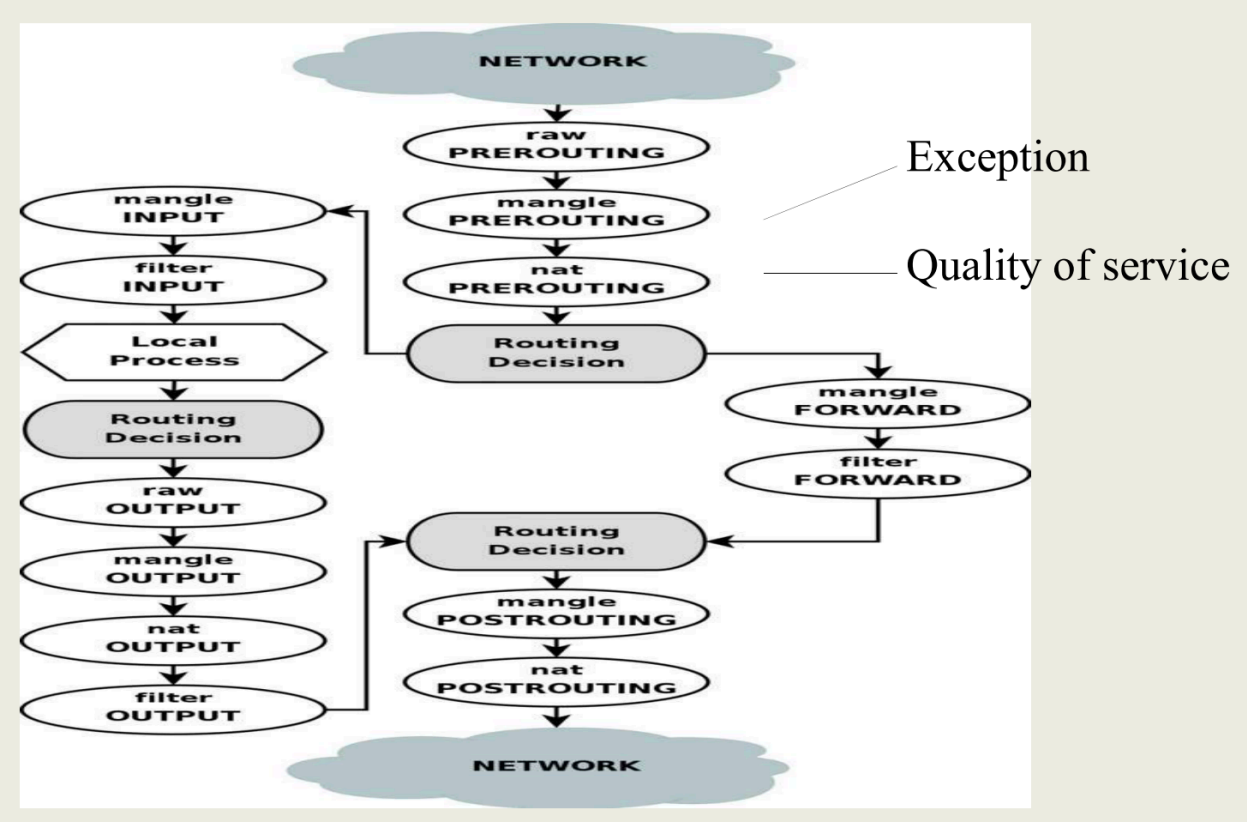
- **Reject**: rifiuto con notifica.
- **DNAT/SNAT/Masquerade**: modifica degli indirizzi IP.

Tabella NAT

- **Prerouting chain**: modifica pacchetti in ingresso prima dell'inoltro.
- **Postrouting chain**: modifica pacchetti in uscita.
- Il NAT cambia gli indirizzi nei pacchetti prima dell'INPUT e dopo OUTPUT/FORWARD.

```
# Abilita il masquerading per la connessione a internet
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

# Redirige il traffico dalla porta 8080 alla porta 80
sudo iptables -t nat -A PREROUTING -p tcp --dport 8080 -j
REDIRECT --to-port 80
```



Esempi

Aggiunta una regola che cancella tutti i pacchetti UDP

```
iptables –A INPUT –p UDP drop
```

Scarta ogni pacchetto TCP indirizzato alla porta 156

```
iptables –A INPUT –p TCP –dport 156 drop
```

Permetti traffico web

```
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

Permetti traffico HTTPS

```
iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

Crea una nuova catena dove poter inserire nuovi

```
iptables – N newcontrol
```

SE - Linux

SE Linux (Security-Enhanced Linux) è una versione di Linux sviluppata dalla NSA (National Security Agency) per migliorare la sicurezza del sistema operativo, in particolare per quanto riguarda il controllo e la gestione degli accessi.

SE Linux introduce un controllo di accesso avanzato che permette di implementare politiche di sicurezza sia basate su **DAC** (Discretionary Access Control) che su **MAC** (Mandatory Access Control).

Ecco un dettaglio di come funziona:

1. **Politiche di Sicurezza Configurabili**:

   SE Linux permette all'utente di configurare diverse politiche di sicurezza. Questo significa che, a differenza del modello DAC (dove l'accesso alle risorse è determinato dal proprietario dei file), SE Linux fornisce un sistema obbligatorio che può definire politiche più complesse per il controllo degli accessi.

2. **Controllo degli Accessi**:

   Mentre in un sistema Linux tradizionale i diritti di accesso vengono definiti in base a utenti e gruppi, SE Linux estende questo concetto, definendo i diritti non solo per gli utenti, ma anche per i programmi e le risorse di sistema.

3. **Controllo dei Diritti sui Programmi**:

   SE Linux definisce **quali diritti ha ogni programma** (o processo) sulle risorse del sistema. In pratica, ogni processo non ha accesso illimitato a tutte le

risorse del sistema, ma solo alle risorse per le quali ha esplicitamente il permesso.

4. **Definizione di Diritti in Termini di Tipo, Ruolo e Livello**:
   - **Tipo**: Ogni oggetto (file, processo, ecc.) è associato a un "tipo" che definisce il tipo di risorsa (ad esempio, un file di log o un processo di rete).
   - **Ruolo**: I ruoli vengono utilizzati per determinare quali operazioni possono essere eseguite da un determinato programma o utente su un determinato tipo di risorsa. Un ruolo potrebbe essere un "utente normale" o un "superuser", ad esempio.
   - **Livelli**: Il controllo basato sui livelli è ortogonale rispetto ai concetti di tipo e ruolo, e si riferisce alla classificazione della sensibilità delle risorse, simile alla gestione della segretezza in un sistema militare (ad esempio, livello di segretezza "confidenziale" o "top secret").

Esempio di Politiche:

- **type1** può eseguire un'operazione su **type2**: Un programma di tipo1 può avere il permesso di leggere o scrivere su una risorsa di tipo2.
- Un programma che un utente può eseguire è determinato dalla politica di SELinux, che associa i permessi a specifici tipi di programmi e risorse.
- **Controllo sul livello**: Il livello di sicurezza ortogonale è usato per determinare se un processo può interagire con una risorsa che ha un determinato livello di sicurezza.

```
allow user_t bin_t : file { read execute write getattr setatt
r };
```

`allow`: Questa parola chiave indica che il permesso che stiamo definendo è consentito. Quindi stiamo esprimendo una permissività su una risorsa specifica.

`user_t`:
Questo è il tipo del soggetto, che in SE Linux può essere un processo, un programma, o un utente.
In questo caso, user_t si riferisce a un utente o processo che appartiene al tipo "user_t". Questo potrebbe essere, per esempio, un utente con privilegi normali nel sistema.

`bin_t`: Questo è il tipo dell'oggetto a cui si applica la politica di accesso. In questo caso, bin_t si riferisce a file che sono considerati "binari", tipicamente eseguibili, come programmi o script.

`file`: Questo indica che l'oggetto a cui si riferisce la politica di accesso è di tipo file. In altre parole, stiamo definendo i permessi relativi ai file nel sistema.

`{ read execute write getattr setattr }`:

Questi sono i diritti che il soggetto (user_t) può esercitare sull'oggetto (bin_t). Ogni verbo rappresenta una specifica operazione che il processo può fare sul file:

`read`: Il processo può leggere il file.
`execute`: Il processo può eseguire il file (essenziale per i file binari o gli script).
`write`: Il processo può scrivere (modificare) il file.
`getattr`: Il processo può ottenere le informazioni sugli attributi del file (ad esempio, i permessi, il proprietario, le dimensioni).
`setattr`: Il processo può modificare gli attributi del file (come i permessi o la data di modifica).